



Security Assessment Final Report



Compound Treasury Manager

April 2026

Prepared for Compound

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Compound Treasury Manager	https://github.com/wooof-software/compound-treasury-management/tree/CU-86c94uxbm-treasury-mvp	ccd20d888ff8637a5b93c3a0b58aaa888e27daaa be60c80a61cb3056802712ec21ce02646613df81	Solidity

Project Overview

This document describes the security review of **Compound Treasury Manager**. The work was undertaken from **April 13th, 2026** to **April 15th, 2026**.

The following contract list is included in our scope:

compound-treasury-management/contracts/interfaces/ICompoundTreasury.sol
compound-treasury-management/contracts/interfaces/ICompoundTreasuryCore.sol
compound-treasury-management/contracts/lib/CompoundTreasuryErrors.sol
compound-treasury-management/contracts/lib/CompoundTreasuryEvents.sol
compound-treasury-management/contracts/lib/CompoundTreasuryTypes.sol
compound-treasury-management/contracts/CompoundTreasury.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

CompoundTreasury.sol is a smart contract designed to hold treasury funds, which allows whitelisted parties to manage the spend of such funds according to predefined criteria, such as:

- **tmms**: treasury manager multisig, the party allowed to initiate or cancel a withdrawal
- **cooldown**: the mandatory wait for a withdrawal requested by the **tmms** to be executed
- **tsg**: treasury safe guard, the party authorized to block, pause or unpause withdrawals

The **CompoundTreasury.sol** contract has been deployed with the following addresses on Ethereum Mainnet:

Contract	Address
CompoundTreasury.sol	0xDcB34b56842F853A69E86De5A0c22c49d97C130C
Treasury Management Multisig (TMMS)	0x8BacCcA3e83843555867b3865d7689D099406567
Treasury Safe Guard (TSG)	0xbbf3f1421D886E9b2c5D716B5192aC998af2012c

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

`CompoundTreasury.sol` is a system to manage treasury funds. It has a `tmms`, which can move any amount of any currency held by the contract, as long as the `cooldown` period is respected between initiating and executing the transfer. A protective `tsg` balances this, with authority to block any initiated withdrawal, or to pause all withdrawals (with the exception of transfers done directly by the `TIMELOCK` via `revoke()`).

In the audit, besides ensuring the logic is sound, we have ensured the access control is properly set and cannot be bypassed, and the transfer power boundaries are always respected. For example, we checked:

- Can anyone besides the expected party (`tmms`) initiate a withdrawal?
- Can anyone besides the expected parties (`tmms`, `tsg`, `TIMELOCK`) stop a withdrawal?
- Can any withdrawal be executed when the system is supposed to be paused?
- Can anyone besides the expected party (`TIMELOCK`) set sensitive variables such as `tmms`, `tsg`, `cooldown`, `expiration`?
- Can the `TIMELOCK` be prevented from executing `revoke()`?

Our conclusion is that the answer to each of these questions is no, which means the underlying safety properties are held; besides 2 low-severity and 2 informational-severity issues, the system does what it is supposed to do and does not have any major risks.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	2	2	0
Informational	2	2	2
Total	4	4	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
L-01	Withdrawals follow the cooldown and expiration settings from when it is initiated, not executed	Low	Acknowledged
L-02	TIMELOCK has limited power	Low	Acknowledged
I-01	Maximum cooldown cannot be set	Informational	Fixed
I-02	The same withdrawal can be canceled or blocked and then executed by reentrance	Informational	Fixed

Low Severity Issues

L-01: Withdrawals follow the cooldown and expiration settings from when it is initiated, not executed

Severity: Low	Impact: Low	Likelihood: Medium
Files: CompoundTreasury.sol	Status: Acknowledged	

Description:

The `CompoundTreasury.sol` contract has `cooldown` and `expiration` state variables that define, respectively, the earliest and latest time a withdrawal must be executed, after being initiated.

These settings can be changed by the `TIMELOCK`. However, the updated settings are only applied to withdrawals initiated after the update, and not to those initiated before.

This causes a delayed settings update.

Recommendations:

Consider querying the `cooldown` and `expiration` variables by the time the withdrawal is completed, not when it is initiated.

Customer Response:

Expected behavior. It reflects the initial requirements – Timelock can change the cooldown for new withdrawals only, and each withdrawal proposal has a fixed in time cooldown and expiration

L-02: TIMELOCK has limited power

Severity: Low	Impact: Medium	Likelihood: Low
Files: CompoundTreasury.sol	Status: Acknowledged	

Description:

The **TIMELOCK** address is the maximum authority for the **CompoundTreasury**, it can:

- Set and remove **tmms** and **tsg**
- Set withdrawal **cooldown** and **expiration**
- Pull all the funds from the contract (**revoke()**)
- Initiate withdrawals
- Pause or unpause withdrawals

However, the **TIMELOCK** is a slower operator, and has an important constraint in its authority:

- It cannot block or cancel a withdrawal
- It can pause all withdrawals, but the **tsg** can unpause them

So there is a scenario where a compromised **tsg** can overpower the **TIMELOCK** and allow a withdrawal request to go through, if the **TIMELOCK** can't move fast enough.

Recommendations:

Allow the **TIMELOCK** to cancel or block a withdrawal; or

Remove the **tsg** access to unpause withdrawals

Customer Response:

Expected behavior. By the nature of the Timelock it is a slow agent, since any its action requires proposal approved and voted by DAO, and that takes around a week of voting/execution time. That is why the contract has TSG which acts as a trusted limited authority which cancel and

pause proposals. In the accepted trust model we accepted TSG as a trusted agent. So TSG is trusted with proposals `unpause()` as an optimization in process where DAOs actions are slow.

As for the TSG compromise, we have the next response plan:

- DAO has the control over roles changing
- DAO can pause proposals execution
- so in case of TSG compromise we issue a proposal which combines 2 actions: change of the TSG and proposals pause.
- treasury stays paused until the expiration runs out on the issued proposals or new TSG cancels them
- treasury continues work with new TSG.

Also, we patched the behavior a bit: removed Timelock from `initiateWithdraw()` method, since it is TMMS-only method, and Timelock follows own `revoke()` method.

Informational Severity Issues

I-01: Maximum cooldown cannot be set

Files:

CompoundTreasury.sol

Description:

`CompoundTreasury.sol` sets the constant variables:

```
uint40 public constant MAX_COOLDOWN = 30 days;  
uint40 public constant MAX_EXPIRATION = 30 days;
```

These are used to bound, respectively, the maximum `cooldown` and `expiration` variables.

However, another condition prevents `MAX_COOLDOWN` to ever be applied to `cooldown`: the contract enforces `expiration > cooldown`.

Considering the expiration must be strictly greater than the cooldown, and the maximum expiration is the same as the maximum cooldown, that will prevent the maximum cooldown time to ever be applied.

Recommendations:

If the team ever intends to set a cooldown period of exactly 30 days, then the maximum expiration time should be increased by at least 1 second; otherwise, the implementation can be kept as is, given this is mostly harmless.

Customer Response:

Agreed. We will set `MAX_EXPIRATION` to 32 days, giving the possibility to set expiration for 48 hours more in case of maximum cooldown set.

Fix Review:

Fix confirmed on [commit 96f603c](#)

I-02: The same withdrawal can be canceled or blocked and then executed by reentrance

Files:

CompoundTreasury.sol

Description:

Once the `tmms` or the `TIMELOCK` initiates a withdrawal via `CompoundTreasury::initiateWithdrawal()`, it is registered with a `WithdrawalStatus.Pending` status.

Then, it can be:

- **Executed** permissionlessly after the contract's `cooldown`, which sends the funds and emits a `WithdrawalExecuted` event
- **Canceled** by the `tmms`, which prevents the withdrawal to be executed and emits a `WithdrawalCancelled` event
- **Blocked** by the `tsg`, which prevents the withdrawal to be executed and emits a `WithdrawalBlocked` event

Each of these statuses should be terminal.

However, the design of `initiateWithdrawal()` allows a withdrawal for a reentrant token to transit from a canceled or blocked state to executed:

```
function executeWithdrawal(uint256 _requestId) external nonReentrant {
    require(!isWithdrawalPaused, CompoundTreasuryErrors.WithdrawalsPaused());
    CompoundTreasuryTypes.WithdrawalRequest memory request =
withdrawalRequests[_requestId];
    require(request.amount != 0,
CompoundTreasuryErrors.WithdrawalRequestNotFound());
    require(
@1>        request.status == CompoundTreasuryTypes.WithdrawalStatus.Pending,
        CompoundTreasuryErrors.WithdrawalRequestNotPending()
    );
    require(block.timestamp > request.cooldown,
CompoundTreasuryErrors.WithdrawalRequestNotReady());
    require(block.timestamp < request.expiration,
```

```
CompoundTreasuryErrors.WithdrawalRequestExpired());
@2>    uint256 balance = request.asset == ETH ? address(this).balance :
IERC20(request.asset).balanceOf(address(this));
        require(balance >= request.amount,
CompoundTreasuryErrors.InsufficientBalance());

@3>    withdrawalRequests[_requestId].status =
CompoundTreasuryTypes.WithdrawalStatus.Executed;

        request.asset != ETH
            ? IERC20(request.asset).safeTransfer(request.recipient,
request.amount)
            : Address.sendValue(payable(request.recipient), request.amount);

        emit CompoundTreasuryEvents.WithdrawalExecuted(_requestId, request.asset,
request.amount, request.recipient);
    }
```

- @1: First, the function validates the status
- @2: Then, it makes an external call to the asset's `balanceOf()` function
- @3: Then, it updates the withdrawal status and proceeds to complete the transfer

With a reentrant token through `balanceOf()`, a `tmms` or `tsg` could reenter the contract and call `cancelWithdrawal()` or `blockWithdrawal()` during step 2.

Those functions would execute, since the withdrawal status hasn't yet been updated.

Then, the function would resume the execution and update the withdrawal status from canceled or blocked to executed.

Recommendations:

Update the status prior to making the external call; or

Add reentrant guard to `cancelWithdrawal()` and `blockWithdrawal()`

Customer Response:

Quite a rare case, possible mechanicy, though not operationally. However balance checks were added as sanity checks for the sake of readable error only – ERC20.transfer and sendValue() will revert in case of insufficient balance anyway. We removed those checks in a latest patch to simplify the code.

Fix Review:

Fix confirmed on [commit 19c275d](#)

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.